Positional Systems

沈威宇

November 8, 2026

Contents

1 Positional Systems 1

I Definition 1

II Conversion between bases 1

i General conversion algorithm 1

ii Power-base conversion algorithm 2

iii Root-base conversion algorithm 2

iv Product and quotient-base conversion algorithm 3

v Criterion of finite expansion after base conversion 3

1 Positional Systems

I Definition

Positional systems, positional numeral system, place-value notations, or place-value, usually denoted as extension to any base of the Hindu–Arabic numeral system, are classified by their base or radix, which is the number of symbols called digits used by the system. These systems in Hindu–Arabic numeral system are also called number system.

Let the base (aka radix) $b \in \mathbb{N}_{>1}$. A base-b (aka radix-b) number

$$N = \left((-1)^m \dots d_2 d_1 d_0 . d_{-1} d_{-2} \dots\right)_b, \quad m \in \{0, 1\} \wedge \forall i : d_i \in \mathbb{N}_0 \wedge d_i < b$$

represents the value

$$N = (-1)^m \sum_i d_i b^i.$$

We can only represent a base in another base or in decimal (base-10), and we have to specify the base you use if not in decimal; otherwise, every base would be base-10.

To indicate the base of a base-b number, we append subscript $_b$ after it. We also use prefix `0b` (or `0B`) to represent base-2 numbers, aka binary numbers, prefix `0` (or `0o`, `0O`) to represent base-8 numbers, aka octal numbers, prefix `0x` (or `0X`) to represent base-16 numbers, aka hexadecimal number, and no prefix (or `0d`, `0D`) to represent base-10 numbers, aka decimal numbers.

Usually, when the radix $b > 10_{10}$, A (or a) is used to represent the digit $9 + 1$; when the radix $b > 11_{10}$, B (or b) is used to represent the digit $9 + 2$; and so on.

II Conversion between bases

i General conversion algorithm

We want to convert a number N in base x with $x \in \mathbb{N}_{>1}$,

$$N = ((-1)^n \dots d_2 d_1 d_0 . d_{-1} d_{-2} \dots)_x, \quad m \in \{0, 1\},$$

into base $y \neq x$ (written in base x) with $y \in \mathbb{N}_{>1}$.

1. First, we take $I_1 = (\dots d_2 d_1 d_0)_x$ and repeatedly:
   (a) take $r_i = I_i \mod y$, convert it to base y, of which the result must be a single digit, and
   (b) take $I_{i+1} = \dfrac{I - r_i}{y}$,

in the ith (start from 1st) time, until the first time $I_{i+1} = 0$, and let that time be the kth time.

2. Second, we take $J_1 = (0.d^{-1}d^{-2}\ldots)_x$ and repeatedly

   (a) take $s_j = \lfloor J_j \cdot y \rfloor$, which may be more than 1 digits if $x > y$ and is later treated directly as in base y, and

   (b) take $J_{j+1} = J_j \cdot y - s_j$,

   in the jth (start from 1st) time, until the first time $J_{j+1} = 0$, and let that time be the lth time.

3. Then, N is

$$(-1)^n \left(\sum_{i=1}^{k} r_i \times y^{i-1} + \sum_{j=1}^{l} s_j \times y^{-j} \right),$$

   and N in base y can be obtained by first taking the integer part of the magnitude of it:

$$(r_k r_{k-1} \ldots r_2 r_1)_y$$

   and then obtaining the fractional part of the magnitude of it using vertical addition where the ith row is s_i with the units digit of s_i in the ith digit to the right of the radix point, for each i, where carry propagation is applied to ensure all digits are valid in base y, i.e. $< y$.

This conversion algorithm is $O(m \log m)$, where m is the total number of digits.

To convert an expression that is a radix-independent fuction f of a vector of numbers $(p_1, p_2, \ldots)$, we convert each p_i to the wanted base, let it be q_i, then the wanted result is $f(q_1, q_2, \ldots)$.

ii Power-base conversion algorithm

To convert a base-b number into base b^n with $b, n \in \mathbb{N}_{>1}$, we group the digits to the left and right of the radix point into blocks of n digits starting from the digits closest to the radix point and convert each block to one digit in base b^n. This algorithm method is $O(m)$, where m is the total number of digits.

iii Root-base conversion algorithm

To convert a base-b^n number in into base b with $b, n \in \mathbb{N}_{>1}$, we convert each digit to n digits in base b and concatenate them. This algorithm method is $O(m)$, where m is the total number of digits.

iv Product and quotient-base conversion algorithm

We want to convert a number in base x with $x \in \mathbb{N}_{>1}$,

$$N = ((-1)^m \dots d_2 d_1 d_0 . d_{-1} d_{-2} \dots)_x, \quad m \in \{0, 1\},$$

into base $y = ax$ (written in base x) with $y \in \mathbb{N}_{>1}$.

We take $D_i = d_i \cdot a^{-i}$, which is later treated directly as in base y, for each integer i.

Then, N is

$$(-1)^m \sum_i D_i \times y^i,$$

and N in base y can be obtained by vertical addition where the ith row is D_i with the units digit of D_i in the $(i+1)$th digit to the left of the radix point, for each i, where carry propagation is applied to ensure all digits are valid in base y, i.e. $< y$.

This conversion algorithm is $O(m)$, where m is the total number of digits.

v Criterion of finite expansion after base conversion

A finite rational number with the fractional part of the magnitude of it expressed in fraction in base x with $x \in \mathbb{N}_{>1}$ being D_x can be expressed in finite digits in radix point in base $y \neq x$ with $y \in \mathbb{N}_{>1}$ iff the reduced denominator (denominator cancelled the greatest common factor with the numerator) of D_x has no prime factor that is not a prime factor of y.